

≡ VERIQTA ≡

Infrastructure
as Code.
Reliability as
Standard.

TERRAFORM

PRODUCTION HANDBOOK

VOLUME 1

- ✓ Real-World Patterns
- ✓ Production Best Practices
- ✓ Troubleshooting Guides
- ✓ Interview Insights
- ✓ Mini Projects

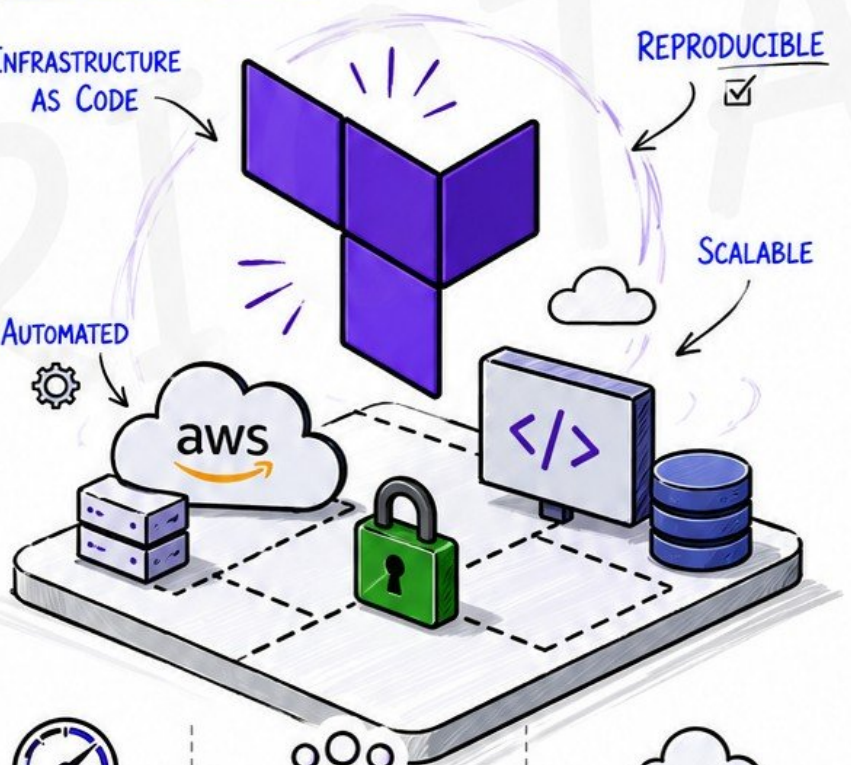
Everything you need
to master **Terraform**
in **Production!**

INFRASTRUCTURE
AS CODE

REPRODUCIBLE

AUTOMATED

SCALABLE



WRITE ONCE

Use code to build
reliable infrastructure.



RELIABLE

Consistent, secure
and repeatable
deployments.



SCALABLE

Designed for growth
and production
workloads.



TEAM READY

Collaboration, version
control and remote
state.



CLOUD AGNOSTIC

Works across AWS,
Azure, GCP and
more.

≡ CODE IT. PLAN IT. APPLY IT. OWN IT. ≡



Instagram:
@veriqta



Linkedin:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA

1. INTRODUCTION TO TERRAFORM



Terraform helps you build, change, and version cloud infrastructure safely and consistently.

1. WHAT IS TERRAFORM?



Terraform is an open-source tool by HashiCorp that lets you define and manage infrastructure using code.

You write configuration files, and Terraform provisions and manages your resources across cloud providers.

2. INFRASTRUCTURE AS CODE (IAC)



- ✓ Define infrastructure in code.
- ✓ Version-controlled and repeatable.
- ✓ Automated and consistent.
- ✓ Reviewable and auditable.
- ✓ Reusable across environments.

3. WHY TEAMS USE TERRAFORM



Collaboration: Teams can work on infrastructure together using version control.



Consistency: Same code = same infrastructure every time.



Speed: Automate provisioning and changes in minutes.



Visibility: See planned changes before they are applied.



Cost Control: Manage resources efficiently and avoid waste.

4. TERRAFORM VS MANUAL PROVISIONING

TERRAFORM (IAC)	MANUAL PROVISIONING
✓ Code-based and repeatable	✗ Click-based and error-prone
✓ Version controlled (Git)	✗ No version history
✓ Automated and fast	✗ Slow and manual
✓ Consistent across environments	✗ Drift and inconsistency
✓ Auditable and reviewable	✗ Hard to audit
✓ Safe with plan & approval	✗ Risk of mistakes

5. PRODUCTION USE CASES



Multi-Cloud Deployments

Manage resources across AWS, Azure, GCP, and other providers.



Environment Management

Provision dev, test, staging, and production environments consistently.



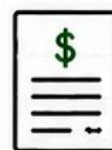
CI/CD Automation

Integrate with pipelines to automate infrastructure changes safely.



Disaster Recovery & Failover

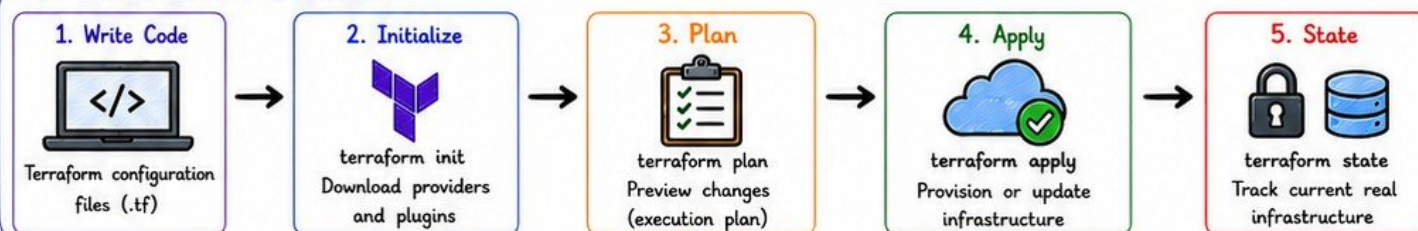
Automate DR environments and failover infrastructure quickly.



Cost Optimization

Manage and right-size resources to reduce cloud costs.

6. ARCHITECTURE OVERVIEW



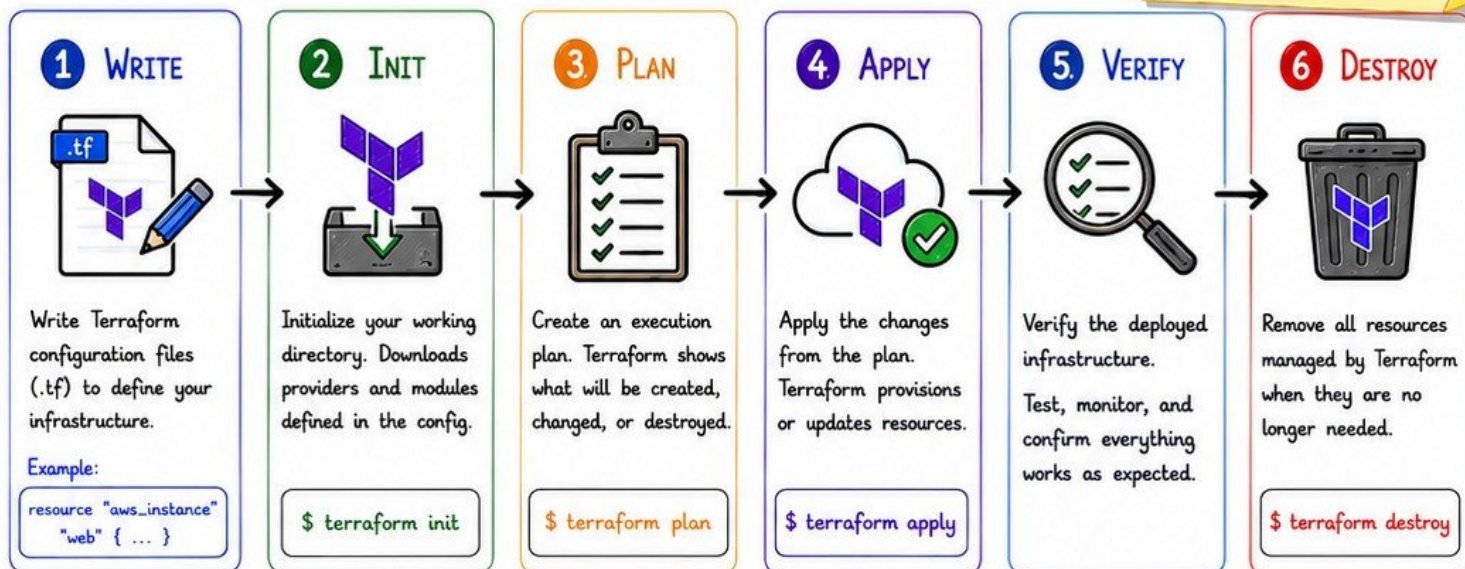
Terraform ensures your desired infrastructure matches the real infrastructure. State file acts as the source of truth.

Code → Plan → Apply → Verify

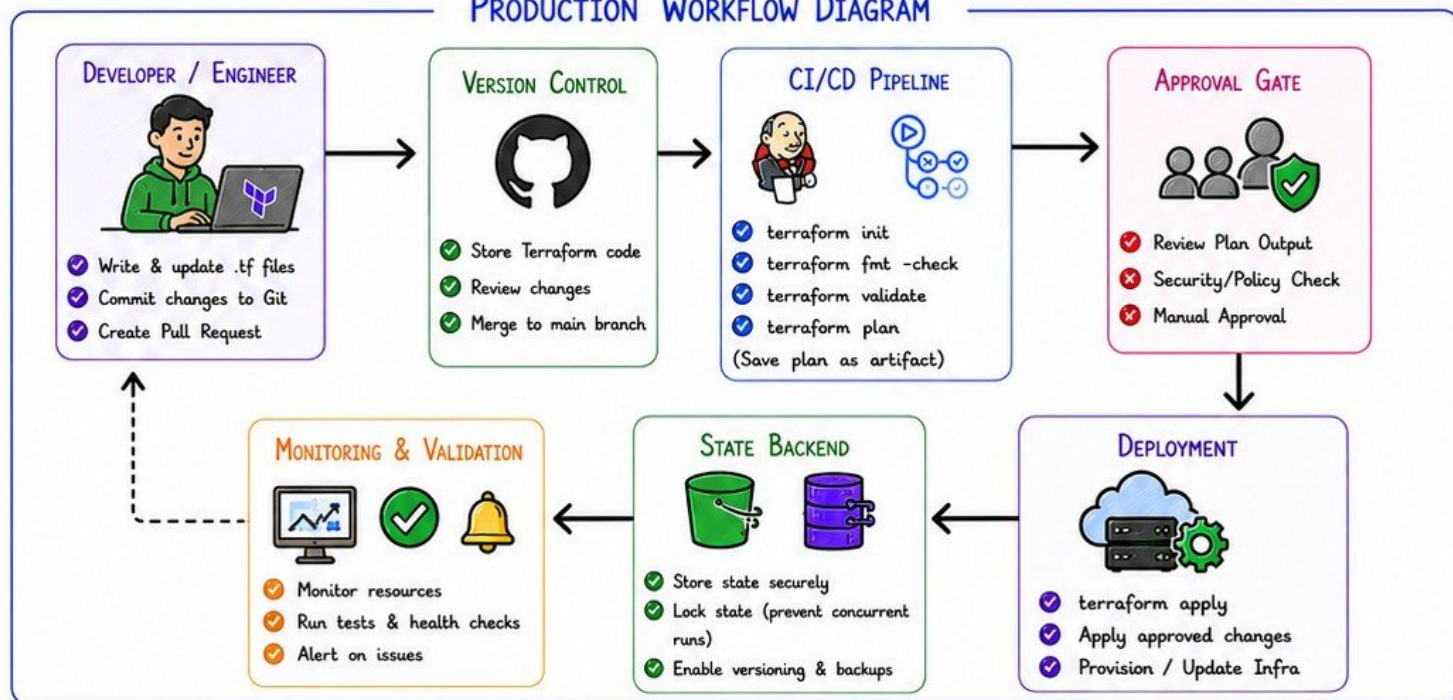
VERIQTA

2. TERRAFORM WORKFLOW

 Terraform follows a simple but powerful workflow to manage infrastructure as code.



PRODUCTION WORKFLOW DIAGRAM



KEY TAKEAWAYS

- ✓ Terraform workflow is repeatable and predictable.
- ✓ Always run PLAN before APPLY.
- ✓ Use version control and CI/CD for safety.
- ✓ State is the source of truth.
- ✓ Validate and verify after every change.

COMMON MISTAKES



- ✗ Applying without reviewing plan.
- ✗ Not storing state remotely.
- ✗ Skipping code review.
- ✗ Destroying resources by mistake.
- ✗ Running multiple applies at the same time.

BEST PRACTICES



- ✓ Use remote state with locking.
- ✓ Store code in Git and use PRs.
- ✓ Automate validation in CI/CD.
- ✓ Review plan carefully.
- ✓ Monitor and alert in production.

VERIQTA

3. PROVIDERS

Providers are plugins that let Terraform communicate with cloud platforms and APIs to manage resources.



1. WHAT ARE PROVIDERS?



Providers are plugins that act as a bridge between Terraform and the APIs of a cloud platform or service.

They allow Terraform to create, update, and manage resources.

EXAMPLE

The AWS provider lets Terraform talk to AWS and manage EC2, S3, RDS, VPC and more.

2. MAJOR CLOUD PROVIDERS

AWS PROVIDER

HashiCorp AWS Provider



Manages AWS resources like EC2, S3, RDS, Lambda, VPC, IAM, etc.

AZURE PROVIDER

HashiCorp Azure Provider



Manages Azure resources like VM, Storage, VNet, App Service, AKS, Key Vault, etc.

GOOGLE PROVIDER

HashiCorp Google Provider



Manages GCP resources like Compute Engine, Cloud Storage, VPC, Cloud SQL, etc.

3. PROVIDER CONFIGURATION

Providers are configured in the terraform {} block.

AWS EXAMPLE

```
provider "aws" {  
  region = "us-east-1"  
  profile = "production"  
}
```

★ Uses AWS CLI profile or environment variables.

AZURE EXAMPLE

```
provider "azurerm" {  
  features {}  
}
```

★ Uses Azure CLI login or Service Principal.

GOOGLE EXAMPLE

```
provider "google" {  
  project = "my-project"  
  region = "us-central1"  
}
```

★ Uses gcloud auth or Service Account key.

AUTHENTICATION METHODS

- ✓ Environment Variables (AWS_ACCESS_KEY_ID, etc.)
- ✓ CLI Credentials (aws configure, az login, gcloud auth)
- ✓ Service Principal / IAM Role
- ✓ Secrets Managers (SSM, Key Vault)

4. VERSION CONSTRAINTS

Always pin provider versions to avoid unexpected breaking changes.

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0" # Accepts 5.0.x but not 6.0.0  
    }  
    azurerm = {  
      source = "hashicorp/azurerm"  
      version = "~> 3.80"  
    }  
  }  
  required_version = ">= 1.5.0"  
}
```

COMMON OPERATORS

=	Exact version
!=	Not this version
>	Greater than
>=	Greater or equal
<	Less than
<=	Less or equal
~>	Pessimistic (recommended)

★ Recommendation: Use ~>

5. COMMON MISTAKES

- ✗ Not pinning provider versions
Can break your deployments when new versions release.
- ✗ Hardcoding credentials in code
This is insecure. Use env vars, IAM roles, or secrets managers.
- ✗ Using incorrect region
Resources may be created in wrong region causing issues.
- ✗ Forgetting to initialize providers
Always run: **terraform init**
- ✗ Using old / unsupported providers
Leads to bugs and missing features. Always keep updated.



KEY TAKEAWAYS

- ✓ Providers are plugins that connect Terraform to cloud platforms.
- ✓ Configure providers securely using best practices.
- ✓ Pin versions to ensure stability and predictability.
- ✓ Follow authentication and security guidelines.

VERIQTA

4. RESOURCES

Resources are the building blocks of infrastructure in Terraform.



1. WHAT ARE RESOURCES?



Resources represent real infrastructure objects that Terraform can create, update, or delete.

Examples: EC2 instance, S3 bucket, VPC, Database, Load Balancer, etc.

2. RESOURCE BLOCKS

Create an AWS EC2 instance

```
resource "aws_instance" "web" {  
  ami           = "ami-0c55b159cbf0e1f0"  
  instance_type = "t3.micro"  
  tags = {  
    Name        = "web-server"  
    Environment = "production"  
  }  
}
```

Resource Type

Resource Name

Arguments (Inputs)

3. ARGUMENTS (INPUTS)

Arguments are the configuration values you set for a resource.



Argument	Description
ami	The Amazon Machine Image ID for the EC2 instance
instance_type	Defines the size and compute capacity of the instance
tags	Key-value pairs to label and organize resources

4. ATTRIBUTES (OUTPUTS)

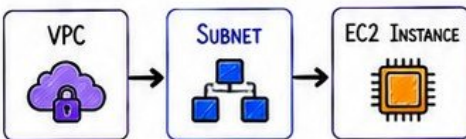
Attributes are read-only values that Terraform learns after the resource is created.

Attribute	Description
id	Unique ID of the resource
arn	Amazon Resource Name
public_ip	Public IP address (for EC2)
status	Current status of the resource



5. DEPENDENCIES

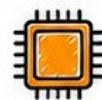
Terraform automatically understands dependencies based on references.



If a resource depends on another, Terraform creates them in the correct order.

6. REAL INFRASTRUCTURE EXAMPLES

EC2 INSTANCE



Virtual server in the cloud to run your applications.

S3 BUCKET



Object storage for files, backups, and static websites.

VPC



Isolated network environment in the cloud.

RDS DATABASE



Managed database service (MySQL, PostgreSQL, etc.)

ALB



Distributes traffic across multiple resources.

★ KEY TAKEAWAYS

- ✓ Resources are real infrastructure objects.
- ✓ Resource blocks define what to create.
- ✓ Arguments are inputs you configure.
- ✓ Attributes are outputs Terraform provides.
- ✓ Terraform handles dependencies for you.



Think of resources as the pieces. Terraform is the engine that builds and manages your infrastructure from code.



Code



Plan



Apply



Real Infrastructure

VERIQTA

5. VARIABLES



Variables make your Terraform configuration flexible, reusable, and environment-agnostic.

1. INPUT VARIABLES



Input variables allow you to parameterize your configurations so you can reuse the same code across different environments (dev, test, prod).

WHY USE VARIABLES?

- ✓ Reuse the same code in multiple environments
- ✓ Make values configurable at runtime
- ✓ Avoid hardcoding values
- ✓ Easier collaboration and safer changes

2. DECLARING VARIABLES

```
variables.tf

variable "region" {
  description = "AWS region to deploy resources"
  type       = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type       = string
  default    = "t3.micro"
}

variable "name_prefix" {
  description = "Prefix for all resource names"
  type       = string
  default    = "myapp"
}
```

REQUIRED VARIABLE (no default)

Must be provided when running Terraform.

OPTIONAL VARIABLE (with default)

Terraform uses this value if none is provided.

3. VARIABLE TYPES

Terraform supports multiple types.

	TYPE	EXAMPLE	DESCRIPTION
T	string	"us-east-1"	Text value
123	number	3	Numeric value
✓	bool	true	True or false
☰	list(string)	["a", "b", "c"]	List of strings
{ }	map(string)	{ env = "prod", team = "devops" }	Map of key/value pairs
📦	object	{ name = "web", size = "t3.micro" }	Complex object

4. DEFAULT VALUES

Set a default value so the variable is optional.

```
variable "instance_type" {
  description = "EC2 instance type"
  type       = string
  default    = "t3.micro"
}
```

★ TIP

Use defaults for non-critical values like tags, instance type, or prefixes.

5. VALIDATION RULES

Validate variable values to prevent invalid input.

```
variable "environment" {
  type       = string
  description = "Deployment environment"
  validation {
    condition = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}
```

🛡️ BENEFIT

Catches wrong values early and prevents failed deployments.

6. SENSITIVE VARIABLES

Mark variables as sensitive to hide them in CLI output and plan files.

```
variable "db_password" {
  description = "Database password"
  type       = string
  sensitive   = true
}
```



NOTE
Sensitive values are still stored in state. Use remote state + encryption.

7. USING VARIABLES

Reference variables in your resources.

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
  tags = {
    Name        = "${var.name_prefix}-web"
    Environment = var.environment
  }
}
```

🚀 GOOD PRACTICE

Keep your configurations DRY (Don't Repeat Yourself).

8. PRODUCTION EXAMPLES

How variables are commonly used in real-world projects.

aws AWS ENVIRONMENTS

- ✓ region
- ✓ instance_type
- ✓ vpc_id
- ✓ subnet_ids
- ✓ tags

Different values for dev, staging, prod.

AZURE ENVIRONMENTS

- ✓ location
- ✓ resource_group
- ✓ vm_size
- ✓ subnet_id
- ✓ admin_password

Parameterize resources for each environment.

GCP ENVIRONMENTS

- ✓ project_id
- ✓ region
- ✓ zone
- ✓ machine_type
- ✓ service_account_email

Use variables for projects and regions.

COMMON USE CASES

- ✓ Environment names
- ✓ Naming prefixes
- ✓ Resource sizes
- ✓ CIDR blocks
- ✓ Feature flags

Make infrastructure flexible and reusable.

TEAM & SCALE

- ✓ Consistent variable naming
 - ✓ Use tfvars files per environment
 - ✓ Store secrets securely
- Enables collaboration and safe scaling.

★ KEY TAKEAWAYS

- ✓ Use types and validation to prevent bad inputs.
- ✓ Avoid hardcoding values in your configurations.
- ✓ Mark secrets as sensitive and store state securely.
- ✓ Use variables + tfvars files for multi-environment deployments.

VERIQTA

6. OUTPUTS



Outputs expose important information from your infrastructure so it can be used by humans or other systems.

1. WHAT ARE OUTPUTS?



Outputs are values that Terraform returns after apply. They show useful information about your infrastructure such as IP addresses, DNS names, or IDs.

WHY USE OUTPUTS?

- ✓ Share important values with your team
- ✓ Use values in other systems or tools
- ✓ Pass values between Terraform modules
- ✓ Reduce manual lookup and human error

3. SENSITIVE OUTPUTS

Mark outputs as sensitive to hide them in CLI output and plans.

```
output "db_password" {  
  description = "Database admin password"  
  value       = aws_db_instance.db.password  
  sensitive   = true  
}
```



Sensitive outputs are still stored in the state file.
Secure your state backend!

5. PRODUCTION USE CASES

	Expose ALB DNS name or Cloudfront domain for your application.
	Expose resource IDs for automation scripts or CI/CD pipelines.
	Share VPC ID, Subnet IDs, or Security Group IDs with other teams or modules.
	Expose ARNs or Role names for IAM integration.
	Use outputs for monitoring/alerting setup (Example: RDS endpoint).

2. OUTPUT BLOCKS

Define outputs using the output block in your configuration.

```
# outputs.tf  
output "instance_public_ip" {  
  description = "Public IP of the EC2 instance"  
  value       = aws_instance.web.public_ip  
}  
output "instance_id" {  
  description = "ID of the EC2 instance"  
  value       = aws_instance.web.id  
}  
output "vpc_id" {  
  description = "VPC ID of the environment"  
  value       = aws_vpc.main.id  
}
```

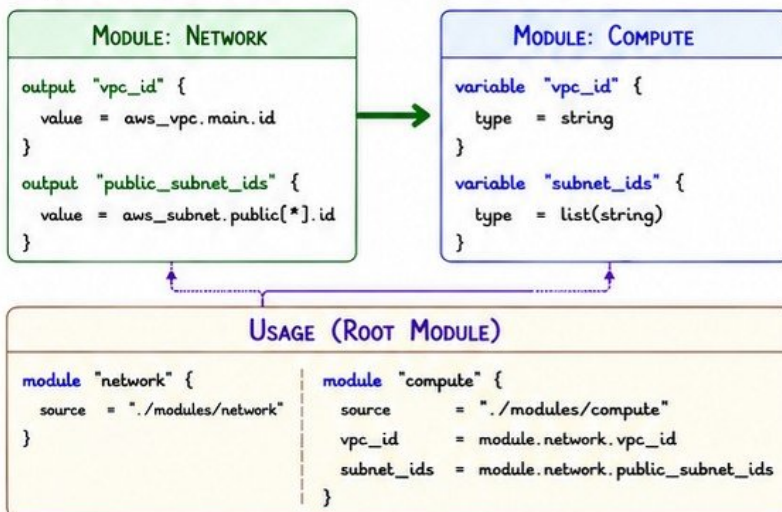
Output Name
(User Defined)

Description
(What it is)

Value
(What it returns)

4. USING OUTPUTS BETWEEN MODULES

Outputs from one module can be used as inputs in another.



6. VIEWING OUTPUTS

After apply, view outputs using:



```
terraform output      # Show all outputs  
terraform output instance_public_ip # Show specific output  
terraform output -json # JSON format
```



NOTE:

- ✓ Outputs do not affect your infrastructure.
- ✓ Outputs are for information sharing and integration.



KEY TAKEAWAYS

- ✓ Outputs return important information after apply.



Use output blocks to expose values.



Mark sensitive values to hide them.



Share outputs between modules to build reusable infrastructure.



Outputs improve automation, collaboration, and visibility.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube : VERIQTA

7. TERRAFORM STATE

State is the memory of your infrastructure. Without state, Terraform can't manage your resources safely.

1. WHAT IS STATE?

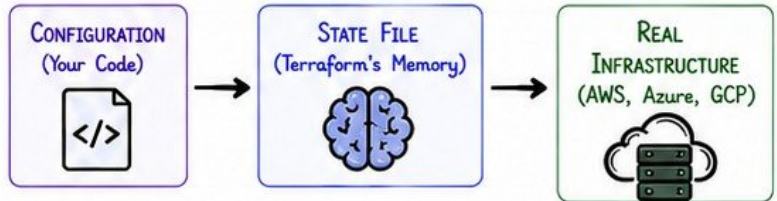


State is a file that Terraform uses to track the real infrastructure it manages. It maps your configuration to the actual resources in the real world.

STATE FILE (terraform.tfstate) CONTAINS:

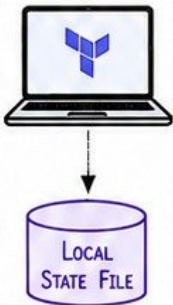
- ✓ Resource IDs (ARN, IDs, IPs, etc.)
- ✓ Resource attributes (last known values)
- ✓ Dependencies between resources
- ✓ Metadata about your infrastructure

2. WHY TERRAFORM NEEDS STATE



- ✓ Terraform compares config with state to know what to create, update, or destroy.
- ✓ It avoids recreating existing resources.
- ✓ It knows dependencies and the correct order of operations.
- ✓ It detects drift (changes made outside Terraform).
- ✓ It enables safe collaboration across teams.

3. LOCAL STATE



By default, Terraform stores state locally on your machine in the working directory.

DEFAULT LOCATION

./terraform.tfstate

✓ PROS

- ✓ Simple setup
- ✓ No extra services needed
- ✓ Great for learning & labs

✗ CONS

- ✗ Not shared across team
- ✗ No locking (risk of conflicts)
- ✗ Not safe for production

4. STATE FILE RISKS



Loss of State File = Disaster

Losing the file can lead to resource recreation, or even failed operations.



Manual Changes = Drift

Changing resources outside Terraform causes drift and unexpected behavior.



Sensitive Data Exposure

State may contain IPs, ARNs, resource names, outputs, and sometimes secrets.



Team Conflicts

Multiple users running Terraform can overwrite each other's changes without locking.



5. STATE LOCKING

State locking prevents multiple operations from modifying the state at the same time.

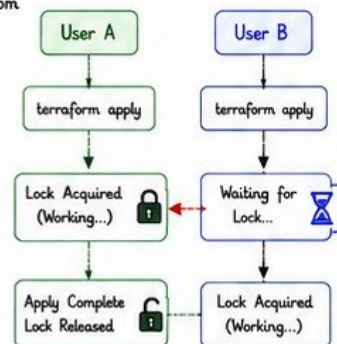


HOW IT WORKS

When a Terraform operation starts, it acquires a lock. Other operations must wait until the lock is released.

BENEFITS

- ✓ Prevents concurrent writes
- ✓ Avoids state corruption
- ✓ Ensures team safety



6. PRODUCTION IMPORTANCE



Single Source of Truth

State is the source Terraform trusts. Keep it safe and always up to date.



Team Collaboration

Use remote state with locking so teams can work in parallel without conflicts.



Disaster Recovery

Back up your state file regularly. It's critical to restore infrastructure.



Automation & CI/CD

Remote state enables secure and repeatable automation pipelines.



Audit & Compliance

State helps you track what was created, when, and by whom.



KEY TAKEAWAYS

- ✓ State is Terraform's memory of your infrastructure.
- ✓ Treat your state file like a production asset.
- ✓ Use remote state with locking in production.
- ✓ Back it up, protect it, and never edit it manually.
- ✓ Good state management prevents outages and saves time.

VERIQTA

8. REMOTE STATE

Remote state stores your Terraform state file in a central, secure location with locking and versioning.

1. WHY REMOTE STATE MATTERS

- ✓ Team collaboration - everyone uses the same state.
- ✓ State locking - prevents concurrent modifications and corruption.
- ✓ Durability - state is backed up and survives machine failures.
- ✓ Security - stored in secure cloud storage with IAM controls.
- ✓ Scalability - supports large teams and multiple environments.

2. S3 BACKEND (AWS)

```
terraform {
  backend "s3" {
    bucket      = "veriqta-terraform-state"
    key         = "prod/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "veriqta-terraform-locks"
    encrypt     = true
  }
}
```

S3 FEATURES



- ✓ Highly Durable (11 9s)
- ✓ Versioning (enable to recover old state)
- ✓ Server-Side Encryption (SSE-S3 / SSE-KMS)

3. DYNAMODB LOCKING



DynamoDB provides state locking to ensure only one operation can modify the state at a time.

HOW IT WORKS

- 1 Terraform tries to acquire a lock.
- 2 If lock is free → acquired.
- 3 Others must wait until lock is released.
- 4 Lock is released after operation completes or on failure.

TABLE EXAMPLE

Table Name	veriqta-terraform-locks
Partition Key	LockID (String)
Billing Mode	On-Demand
TTL (optional)	auto remove stale locks

★ NOTE:

Never disable locking in production. It can corrupt your state.

4. AZURE STORAGE BACKEND

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "veriqtastateprod"
    container_name      = "tfstate"
    key                 = "prod/terraform.tfstate"
    use_azuread_auth    = true
  }
}
```

AZURE FEATURES



- ✓ Redundant Storage (GRS / RA-GRS)
- ✓ Versioning
- ✓ Encryption at Rest
- ✓ Azure AD / RBAC Access Control

5. GCS BACKEND (GOOGLE CLOUD)

```
terraform {
  backend "gcs" {
    bucket      = "veriqta-tf-state-prod"
    prefix      = "prod/terraform.tfstate"
    credentials = "gcp-terraform-sa.json" # or ADC
    project     = "veriqta-prod"
  }
}
```

GCS FEATURES

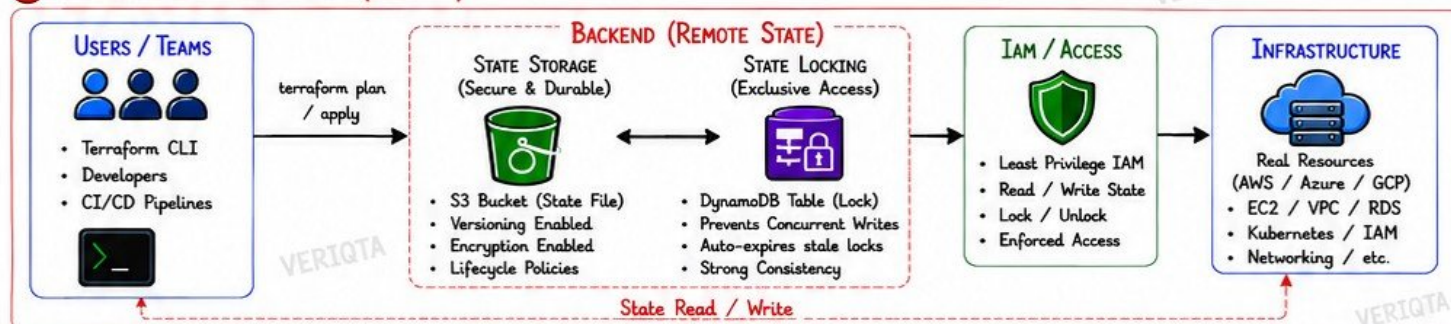


- ✓ Multi-Regional Durability
- ✓ Object Versioning
- ✓ Bucket-Level Encryption
- ✓ Fine-grained IAM

6. COMPARISON

FEATURE	AWS (S3 + DDB)	AZURE (STORAGE)	GCP (GCS)
State Storage	S3 Bucket	Blob Container	GCS Bucket
Locking	DynamoDB	Blob Lease	Built-in Lock
Versioning	Yes	Yes	Yes
Encryption	S3 SSE / KMS	SSE / CMK	CMEK
Auth	IAM Roles	Azure AD / SAS	Service Account
Best For	AWS Environments	Azure Environments	GCP Environments

7. PRODUCTION ARCHITECTURE (EXAMPLE)



★ BEST PRACTICES



Always store state in a remote backend.



Enable locking to prevent corruption.



Enable versioning and test state recovery.



Restrict access with least privilege IAM policies.



Regularly backup state and monitor permissions.



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

9. TERRAFORM COMMANDS

Memorize these commands.
Master them.
Ship with confidence!



COMMAND	PURPOSE	EXAMPLE	WHAT IT DOES
1 terraform init 	Initializes a working directory containing Terraform configuration files. 	<pre>\$ terraform init</pre> Initializing the backend... Initializing provider plugins... Terraform has been successfully initialized!	 ✓ Downloads providers ✓ Configures backend ✓ Prepares the working directory
2 terraform fmt 	Formats Terraform configuration files to a standard style. 	<pre>\$ terraform fmt</pre> main.tf modules/vpc/main.tf outputs.tf (all files formatted)	 ✓ Improves readability ✓ Enforces consistent style ✓ No changes to infrastructure
3 terraform validate 	Validates the Terraform configuration files. 	<pre>\$ terraform validate</pre> Success! The configuration is valid.	 ✓ Checks syntax ✓ Validates structure ✓ Catches configuration errors early
4 terraform plan 	Creates an execution plan showing what Terraform will do. 	<pre>\$ terraform plan</pre> Plan: 2 to add, 1 to change, 0 to destroy. (+) aws_instance.web (~) aws_s3_bucket.logs	 ✓ Shows proposed changes ✓ No changes are applied ✓ Review before apply
5 terraform apply 	Applies the changes required to reach the desired state. 	<pre>\$ terraform apply</pre> Do you want to perform these actions? Enter a value: yes Apply complete! Resources: 2 added, 1 changed, 0 destroyed.	 ✓ Creates/Updates resources ✓ Brings infrastructure in line with configuration ✓ Changes are real and irreversible
6 terraform destroy 	Destroys all resources managed by Terraform. 	<pre>\$ terraform destroy</pre> Do you really want to destroy all resources? Enter a value: yes Destroy complete! ALL resources destroyed.	 ✓ Removes all managed infrastructure ✓ Use with extreme caution
7 terraform show 	Shows the current state or a saved plan in a human-readable format. 	<pre>\$ terraform show</pre> <pre># aws_instance.web: resource "aws_instance" "web" { id = "i-0abc123def456" ... }</pre>	 ✓ Displays state or plan details ✓ Useful for debugging ✓ Great for auditing



PRO TIP

Run these in order:

init

fmt

validate

plan

apply

destroy

Only when you mean it!



Instagram: @veriqta



LinkedIn: VERIQTA



X: @veriqta



YouTube: VERIQTA

VERIQTA

10. TERRAFORM CONFIGURATION STRUCTURE

Good structure makes your Terraform code clean, scalable and production ready!

1 main.tf



The main configuration file. Contains all the resources, data sources and core infrastructure definitions.

```
# main.tf
resource "aws_instance" "web" {
  ami      = "ami-0abcd1234ef56"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
  }
}
```

2 variables.tf



Declares input variables that make your modules flexible and reusable. No hardcoding!

```
# variables.tf
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

3 outputs.tf



Defines output values that expose important information after apply.

```
# outputs.tf
output "instance_ip" {
  value     = aws_instance.web.public_ip
  description = "Public IP of the web server"
}
```

4 providers.tf



Configures the provider (AWS, Azure, GCP, etc.) and authentication settings.

```
# providers.tf
provider "aws" {
  region = "us-east-1"
  profile = "prod"
}
```

5 versions.tf



Locks Terraform and provider versions to ensure consistent and reliable deployments.

```
# versions.tf
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

6 terraform.tfvars



Variable definition file. Stores environment-specific values. Never commit secrets!

```
# terraform.tfvars
instance_type = "t3.small"
env           = "production"
project       = "veriqta-app"
```

7 .terraform.lock.hcl



Auto-generated file that locks provider versions for dependency integrity.

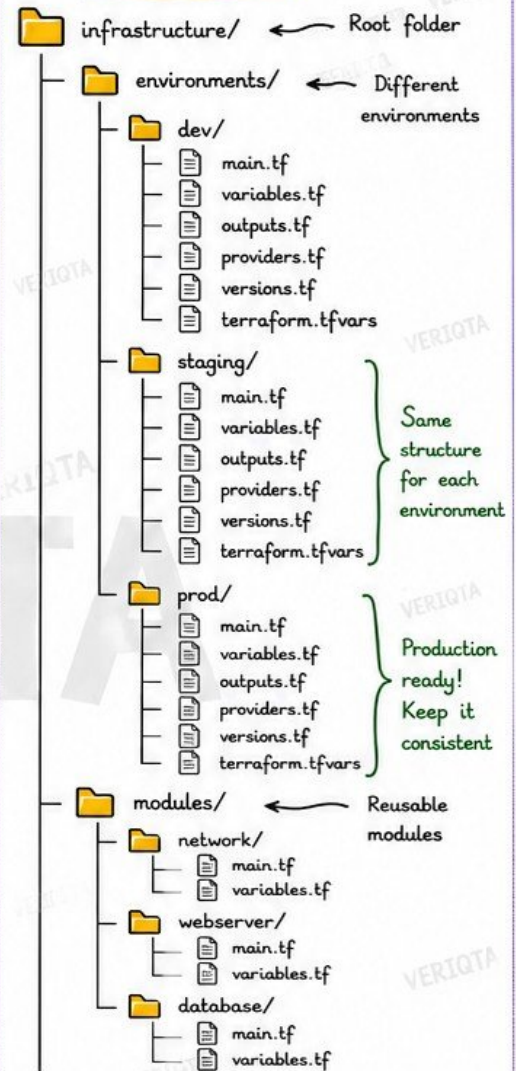
```
# Auto-generated
(Do not edit manually)
```

WHY THIS STRUCTURE MATTERS

- Improves readability and maintainability
- Enables reuse and scalability
- Supports team collaboration
- Reduces errors in production

PRODUCTION FOLDER STRUCTURE

Example Layout



★ BEST PRACTICES

- Use separate folders per environment.
- Keep secrets out of tfvars (use Vault/SSM).
- Commit .tf files, but ignore .tfvars with secrets.
- Use modules for reusability.
- Lock versions to avoid unexpected changes.



PRO TIP

A well-structured Terraform codebase today saves you from production chaos tomorrow!

COMMAND FLOW REMINDER

init → plan → apply → destroy

VERIQTA

11. DATA SOURCES

Data sources let you read existing resources. They connect Terraform to the real world!



1 WHAT DATA SOURCES ARE

- ✓ Data sources are read-only references to existing infrastructure.
- ✓ They fetch information without managing or modifying the resource.
- ✓ They allow Terraform to integrate with infrastructure created outside of Terraform.



2 READING EXISTING INFRASTRUCTURE

Terraform queries the provider API and stores the results in memory during plan/apply.



★ IMPORTANT

Data sources do not create, update or delete anything. They only **READ**. Safe by default!

3 AWS EXAMPLES

Use Case	Data Source	Example
Get VPC	aws_vpc	<pre>data "aws_vpc" "main" { filter { name = "tag:Name" values = ["prod-vpc"] } }</pre>
Get Subnets	aws_subnet	<pre>data "aws_subnet" "private" { filter { name = "tag:Tier" values = ["private"] } }</pre>
Get Security Group	aws_security_group	<pre>data "aws_security_group" "web" { name = "web-sg" vpc_id = data.aws_vpc.main.id }</pre>
Get IAM Role	aws_iam_role	<pre>data "aws_iam_role" "app" { name = "app-role" }</pre>
Get AMI	aws_ami	<pre>data "aws_ami" "ubuntu" { owners = ["099720109477"] filter { name = "name" values = ["ubuntu-22.04-*"] } }</pre>

4 DEPENDENCIES

- ✓ Data sources can depend on variables, locals, or other resources.
- ✓ Terraform automatically resolves the dependency graph.
- ✓ Use `depends_on` only if absolutely necessary.

Example Dependency Flow



5 COMMON MISTAKES

- ✗ Using data sources for resources that Terraform should manage.
- ✗ Assuming data source results are known at plan time (They are, but values can change outside Terraform).
- ✗ Hardcoding IDs instead of using data sources.
- ✗ Not handling missing resources (will cause plan/apply to fail).
- ✗ Forgetting filters and getting multiple or wrong results.



6 PRODUCTION USE CASES

Integrate with Existing Infra Use resources created manually or by other teams.	Cross-Account Lookups Read resources from shared services accounts.	Discover by Tags Dynamically find resources using consistent tagging.	Environment Awareness Adapt configuration based on existing infrastructure.	Reference for New Resources Use data to pass IDs into new Terraform resources.	Compliance & Audit Read and report on existing configuration and settings.
---	---	---	---	--	--

★ PRO TIP

Prefer tag-based lookups over hardcoded IDs. It makes your code **portable, reusable, and environment-agnostic**.

★ KEY TAKEAWAY

Data sources connect Terraform to the existing world safely and intelligently.



Read smart, use tags, and keep your code production-grade!



Read ≠ Manage
Understand the difference!

VERIQTA

12. LOCALS

Locals make your
config cleaner, shorter
and easier to maintain.
Write once, use anywhere!



1 WHAT LOCALS ARE

- ✓ Locals are like variables, but defined inside your configuration.
- ✓ They store values that are computed or used multiple times.
- ✓ They improve readability and help avoid repetition.
- ✓ They are evaluated in memory only (no state stored).



2 REDUCING REPETITION

Without locals (repetitive):

```
resource "aws_s3_bucket" "logs" {  
  bucket = "${var.project}-${var.env}-logs"  
  acl     = "private"  
}  
  
resource "aws_s3_bucket" "assets" {  
  bucket = "${var.project}-${var.env}-assets"  
  acl     = "private"  
}
```

With locals (clean):

```
locals {  
  bucket_prefix = "${var.project}-${var.env}"  
}  
  
resource "aws_s3_bucket" "logs" {  
  bucket = "${local.bucket_prefix}-logs"  
  acl     = "private"  
}  
  
resource "aws_s3_bucket" "assets" {  
  bucket = "${local.bucket_prefix}-assets"  
  acl     = "private"  
}
```

3 CALCULATED VALUES

Locals can perform calculations, string operations, conditionals, lists, maps and more.



```
locals {  
  name      = "${var.project}-${var.env}"  
  region    = var.aws_region  
  environment = lower(var.env)  
  is_prod   = var.env == "prod"  
  common_tags = {  
    Project      = var.project  
    Environment  = title(var.env)  
    ManagedBy    = "Terraform"  
  }  
  cidr_blocks = length(var.public_subnets) > 0  
               ? var.public_subnets  
               : ["10.0.0.0/16"]  
}
```

4 NAMING PATTERNS

PATTERN	EXAMPLE	DESCRIPTION
Lower snake_case	bucket_prefix	Use for all locals
Descriptive names	common_tags	Clear and meaningful
Avoid abbreviations	environment_name	Easy to understand
Group related	vpc_cidr, subnets	Keep things organized
Constants in CAPS	DEFAULT_REGION	For static values



Keep names short but meaningful. Your future self will thank you!

5 PRODUCTION EXAMPLES

COMMON TAGS

```
locals {  
  common_tags = {  
    Project      = var.project  
    Environment  = var.env  
    Owner        = var.owner  
    ManagedBy    = "Terraform"  
    CostCenter   = var.cost_center  
  }  
}
```

→ USE:

Attach to all resources for consistency.



NAME PREFIX

```
locals {  
  name_prefix = "${var.project}-${var.env}-${var.region}"  
}
```

→ USE:

Build standard names across resources.



NETWORK CONFIG

```
locals {  
  vpc_cidr = "10.0.0.0/16"  
  public_subnets = [  
    "10.0.1.0/24",  
    "10.0.2.0/24",  
    "10.0.3.0/24"  
  ]  
  azs = slice(data.aws_availability_zones.available.names, 0, 3)  
  : ["10.0.0.0/16"]  
}
```

→ USE:

Centralize network settings in one place.



CONDITIONAL LOGIC

```
locals {  
  enable_backup = var.env == "prod"  
  instance_type = var.env == "prod"  
    ? "m6i.large"  
    : "t3.micro"  
}
```

→ USE:

Change behavior based on environment.



MERGING MAPS

```
locals {  
  default_tags = {  
    ManagedBy = "Terraform"  
    Source    = "IaC"  
  }  
  tags = merge(local.default_tags,  
    var.additional_tags)  
}
```

→ USE:

Allow overrides while keeping defaults.



KEY TAKEAWAY

- ✓ Use locals to make your code DRY and clean.
- ✓ Compute values once, reuse everywhere.
- ✓ Prefer readability over clever tricks.



Good locals =
Maintainable code
Production confidence!



★ PRO TIP

If you use the same value more than twice, it probably belongs in locals!



VERIQTA

13. EXPRESSIONS & FUNCTIONS

Don't hardcode.
Express smartly.
Automate everything.
Ship confidently!

- ★ Expressions let you build dynamic, flexible and powerful Terraform code. Functions save time and reduce errors.

1 CONDITIONALS

Use condition ? true_value : false_value to make decisions.

```
locals {
  env = var.env == "prod" ? "production" : "non-prod"
  backup_enabled = var.env == "prod" ? true : false
}
```

PRODUCTION EXAMPLE

Enable backups only in production.



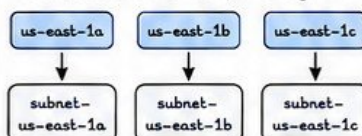
2 FOR EXPRESSIONS

Transform or filter collections using for expressions.

```
locals {
  azs = ["us-east-1a", "us-east-1b", "us-east-1c"]
  subnet_names = [for az in local.azs : "subnet-${az}"]
}
```

PRODUCTION EXAMPLE

Create subnet names for every AZ.



3 LOOKUP

Get a value from a map with a default if the key doesn't exist.

```
locals {
  instance_types = {
    dev = "t3.micro"
    prod = "m6i.large"
  }
  instance_type = lookup(local.instance_types,
    var.env, "t3.small")
}
```

PRODUCTION EXAMPLE

Pick instance type based on environment.



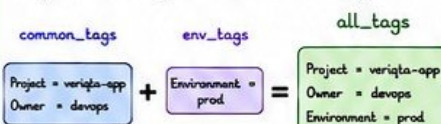
4 MERGE

Combine multiple maps into one.

```
locals {
  common_tags = { Project = "veriqta-app", Owner = "devops" }
  env_tags = { Environment = var.env }
  all_tags = merge(local.common_tags, local.env_tags)
}
```

PRODUCTION EXAMPLE

Merge common tags with environment tags.



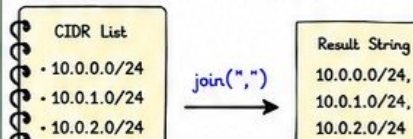
5 JOIN

Join a list of strings into a single string using a delimiter.

```
locals {
  cidrs = ["10.0.0.0/24", "10.0.1.0/24", "10.0.2.0/24"]
  cidr_string = join(" ", local.cidrs)
}
```

PRODUCTION EXAMPLE

Join CIDRs for output or logging.



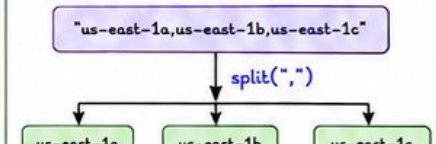
6 SPLIT

Split a string into a list using a delimiter.

```
locals {
  az_string = "us-east-1a,us-east-1b,us-east-1c"
  az_list = split(" ", local.az_string)
}
```

PRODUCTION EXAMPLE

Convert comma-separated AZs into a list.

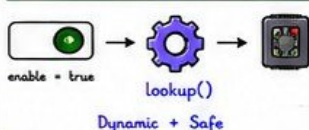


7 PRODUCTION COMBINATIONS (REAL-WORLD POWER)

Conditional + Lookup

Enable feature only for prod and get value from map.

```
locals {
  sizes = { dev = "t3.small", prod = "m6i.large" }
  instance_type = var.enable ? lookup(local.sizes,
    var.env, "t3.micro") : "t3.nano"
}
```



Dynamic + Safe

Merge + For Expressions

Merge tag maps and create resources for each item.

```
locals {
  tags = merge(local.common_tags, {
    Environment = var.env })
  azs = ["us-east-1a", "us-east-1b"]
}
```

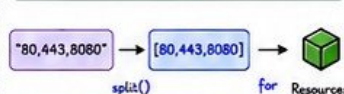


Reusable + Scalable

Split + For Expressions

Split input string and iterate over each item.

```
locals {
  ports_string = "80,443,8080"
  ports = [for p in split(" ", local.ports_string) :
    tonumber(p)]
}
```

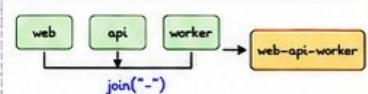


Flexible + Powerful

Join + Output

Join values and output a clean string.

```
locals {
  names = ["web", "api", "worker"]
  names_joined = join("-", local.names)
}
```



Clean Outputs



PRO TIP

Use expressions and functions to make your Terraform code:

✓ DRY
(Don't Repeat)

✓ DYNAMIC
(Environment-aware)

✓ SCALABLE
(Future-ready)

✓ MAINTAINABLE
(Easy to manage)

Smart code ships faster!

VERIQTA

14. COUNT & FOR_EACH

Two powerful ways
to create multiple
resources—use them
wisely!



1 WHY REPETITION MATTERS

- ✓ Reduces manual effort and errors.
- ✓ Makes configurations scalable and maintainable.
- ✓ Ensures consistency across environments.
- ✓ Changes propagate automatically.



2 COUNT

Creates multiple instances of a resource based on a number.

```
resource "aws_instance" "web" {  
  count = 3  
  ami   = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  tags = { Name = "web-${count.index}" }  
}
```

Creates 3 instances: web[0], web[1], web[2]

3 FOR_EACH

Creates multiple instances based on a map or set of values.

```
resource "aws_instance" "web" {  
  for_each = { "a" = "t3.micro", "b" = "t3.small" }  
  ami      = "ami-0abc123def456"  
  instance_type = each.value  
  tags     = { Name = "web-${each.key}" }  
}
```

Creates instances: web["a"], web["b"]

4 DIFFERENCES

Feature	count	for_each
Input Type	Number	Map or Set
Keys	Integers (0..n-1)	Map Keys / Set Values
Reference	count.index	each.key, each.value
Dynamic Changes	Based on count	Based on collection changes
Best For	Simple, indexed resources	Named resources (preferred)

5 RESOURCE ADDRESSING

With count

```
resource[0]  
resource[1]  
resource[2]
```

Indexed by number

With for_each

```
resource["key1"]  
resource["key2"]
```

Indexed by key

★ Addresses are used in outputs, dependencies and references.

6 COMMON MISTAKES

- ✗ Changing count or for_each after creation causes resource recreation.
- ✗ Using both count and for_each in same resource (not allowed).
- ✗ Relying on list indexes with count (not stable).
- ✗ Not using meaningful keys with for_each.
- ✗ Ignoring state impact when keys change.

★ Plan changes before applying!
Review the impact on existing resources.

PRODUCTION EXAMPLES

1. EC2 Instances with count

```
resource "aws_instance" "app" {  
  count = 3  
  ami   = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  tags = { Name = "app-${count.index}" }  
}
```



Creates app[0], app[1], app[2]

2. S3 Buckets with for_each

```
locals {  
  buckets = {  
    logs = "acme-logs-bucket"  
    data = "acme-data-bucket"  
    backup = "acme-backup-bucket"  
  }  
}  
resource "aws_s3_bucket" "buckets" {  
  for_each = local.buckets  
  bucket = each.value  
}
```



Creates buckets: logs, data, backup

3. Security Groups with count

```
resource "aws_security_group" "sg" {  
  count = 2  
  name = "web-sg-${count.index}"  
  vpc_id = var.vpc_id  
  ingress { from_port = 80 to_port = 80 }  
  ingress { from_port = 443 to_port = 443 }  
}
```



Creates sg[0] and sg[1]



PRO TIP

- ✓ Use count for simple numeric repetition.
- ✓ Use for_each for named resources.
- ✓ Always plan and review before apply.



Design for scale.
Build for production.
Sleep better!

REMEMBER

Right tool.
Right structure.
Reliable Infrastructure.



KEY TAKEAWAY

count for numbers, for_each for labels—
both help you build at scale!

Plan



Review



Apply



Scale

VERIQTA

15. MODULES

Modules turn complexity into simplicity. Build once, use everywhere!

1 WHAT MODULES ARE

- ✓ Modules are containers for multiple resources used together.
- ✓ They help you organize, reuse and standardize Terraform code.
- ✓ They make large configurations clean, DRY and easier to maintain.
- ✓ Modules can be local or from remote registries.



2 ROOT MODULE

The root module is the main working directory. It calls other modules.

```
root-module/  
├── main.tf          # calls modules  
├── variables.tf     # root variables  
├── outputs.tf       # root outputs  
├── providers.tf  
└── terraform.tfvars
```



This is where you run Terraform commands (init, plan, apply).

3 CHILD MODULES

Child modules contain reusable components (such as VPC, RDS, EC2, etc.).

```
modules/  
├── vpc/  
│   ├── main.tf  
│   ├── variables.tf  
│   └── outputs.tf  
├── ec2/  
│   ├── main.tf  
│   ├── variables.tf  
│   └── outputs.tf  
└── rds/  
    ├── main.tf  
    ├── variables.tf  
    └── outputs.tf
```

Each folder is a separate module with its own logic.



4 INPUTS (VARIABLES)

Modules accept inputs through variables.

```
# modules/vpc/variables.tf  
variable "cidr_block" {  
  description = "VPC CIDR block"  
  type        = string  
}  
  
variable "azs" {  
  description = "Availability Zones"  
  type        = list(string)  
}
```

Example (root module)

```
module "vpc" {  
  source      = "../modules/vpc"  
  cidr_block = "10.0.0.0/16"  
  azs         = ["us-east-1a", "us-east-1b"]  
}
```



Pass values from the root module into the child module.

5 OUTPUTS

Modules return values using outputs.

```
# modules/vpc/outputs.tf  
output "vpc_id" {  
  value = aws_vpc.this.id  
}  
  
output "public_subnets" {  
  value = aws_subnet.public[*].id  
}
```

Example (root module)

```
output "vpc_id" {  
  value = module.vpc.vpc_id  
}  
  
output "public_subnets" {  
  value = module.vpc.public_subnets  
}
```



Outputs can be used by other modules or shown after apply.

6 REUSABILITY

- ✓ Write once, use many times.
- ✓ Standardize infrastructure across teams.
- ✓ Easier updates and version control.
- ✓ Share modules across environments (dev, staging, prod).



DRY =
Don't Repeat
Yourself

MODULE EXAMPLE

```
Root module (main.tf)  
module "vpc" {  
  source      = "../modules/vpc"  
  cidr_block = "10.0.0.0/16"  
  azs         = ["us-east-1a", "us-east-1b"]  
}  
  
module "ec2" {  
  source      = "../modules/ec2"  
  ami         = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  subnet_id   = module.vpc.public_subnets[0]  
  sg_id       = module.security_group.this_id  
}
```

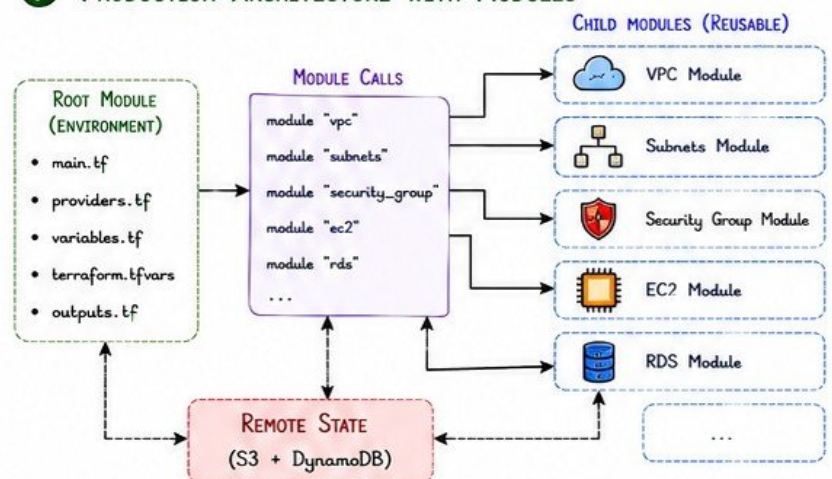
Clean, readable and easy to manage!

COMMON MISTAKES

- ✗ Not using version control for modules.
- ✗ Hardcoding values instead of using variables.
- ✗ Creating overly complex modules that are hard to reuse.
- ✗ Not documenting inputs, outputs and purpose.
- ✗ Breaking changes without versioning.



7 PRODUCTION ARCHITECTURE WITH MODULES



BEST PRACTICES

- ✓ Keep modules small, focused and single-purpose.
- ✓ Use meaningful variable and output names.
- ✓ Add descriptions to all variables and outputs.
- ✓ Version your modules (Git tags or Registry).
- ✓ Test modules independently.



PRO TIP

Use the Terraform Registry or your private Git repo to publish and share modules across teams!

VERIQTA

16. TERRAFORM PROVISIONERS

Provisioners can help, but they can also hurt. Use carefully, and only when absolutely needed!

1 WHAT ARE PROVISIONERS?

- Provisioners run actions on a local machine or on a remote resource after it is created.
- They are defined inside a resource block.
- They are a last resort, not the first choice.



2 LOCAL-EXEC PROVISIONER

Runs a command on the machine where Terraform is running.

```
resource "aws_instance" "web" {  
  ami           = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  
  provisioner "local-exec" {  
    command = "echo 'Instance ${self.id} created' >  
              instance.txt"  
  }  
}
```

Runs on your local workstation



3 REMOTE-EXEC PROVISIONER

Runs commands on the remote resource (after it is created). Uses SSH.

```
resource "aws_instance" "web" {  
  ami           = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  
  provisioner "remote-exec" {  
    inline = [  
      "sudo apt update -y",  
      "sudo apt install -y nginx",  
      "sudo systemctl start nginx"  
    ]  
  }  
}
```



SSH



Terraform Host

Remote Instance

4 FILE PROVISIONER

Copies files or directories from the local machine to the remote resource.

```
resource "aws_instance" "web" {  
  ami           = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  
  provisioner "file" {  
    source      = "../scripts/setup.sh"  
    destination = "/tmp/setup.sh"  
  }  
}
```



Local Machine
(source)



Remote Instance
(destination)

5 WHY PROVISIONERS ARE RISKY

- They break idempotency (Terraform cannot detect changes).
- They run only during create (not on updates by default).
- They can cause "unknown state" in infrastructure.
- Failures can leave resources in a bad state.
- They make configurations harder to debug and maintain.
- They mix configuration with configuration management.



USE
CAREFULLY!

6 BETTER ALTERNATIVES

	SOLUTION	WHEN TO USE
	cloud-init	Initial setup of instances (users, packages, files, services).
	Ansible / Chef / Puppet / Salt	Ongoing configuration management and compliance.
	AWS SSM Run Command	Execute commands or scripts on running instances securely.
	User Data (for simple setup)	Simple bootstrap tasks during instance creation.
	Terraform Modules	Encapsulate and reuse infrastructure logic.

7 WHEN THEY MAY BE USED

- Very simple, one-time tasks after resource creation.
- Copying small files or running basic scripts.
- Bootstrapping when no better option is available.
- Prototyping or quick testing (not for production scale).

GOLDEN RULE

If you can do it without a provisioner, do it. Your future self will thank you!

EXAMPLE: AVOID PROVISIONERS WHEN POSSIBLE

NOT RECOMMENDED (Provisioner)

```
resource "aws_instance" "app" {  
  ami           = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  
  provisioner "remote-exec" {  
    inline = ["sudo apt install -y nginx",  
             "sudo systemctl start nginx"]  
  }  
}
```



RECOMMENDED (cloud-init)

```
resource "aws_instance" "app" {  
  ami           = "ami-0abc123def456"  
  instance_type = "t3.micro"  
  user_data     = <<-EOF  
    #!/bin/bash  
    sudo apt update -y  
    sudo apt install -y nginx  
    sudo systemctl start nginx  
  EOF  
}
```



★ KEY TAKEAWAY

- Provisioners are powerful but risky.
- They are not idempotent.
- Prefer native cloud features and configuration management tools.
- Keep Terraform focused on infrastructure, not configuration.
- Build for reliability and scale!

VERIQTA

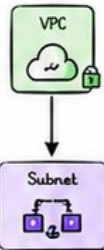
17. TERRAFORM DEPENDENCIES

Terraform builds a dependency graph to create, update and destroy safely!

1 IMPLICIT DEPENDENCIES

- ✓ Terraform automatically understands dependencies by references.
- ✓ If resource A uses an attribute from resource B, A depends on B.
- ✓ No extra configuration needed.

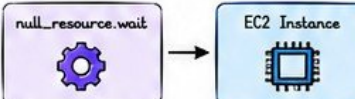
```
resource "aws_vpc" "main" {  
  cidr_block = "10.0.0.0/16"  
}  
  
resource "aws_subnet" "public" {  
  vpc_id = aws_vpc.main.id # Implicit dependency  
  cidr_block = "10.0.1.0/24"  
}
```



2 EXPLICIT DEPENDENCIES

- ✓ Use when Terraform cannot detect the dependency automatically.
- ✓ Common with provisioners, data sources, or resources without direct references.

```
resource "null_resource" "wait" {  
  triggers = {  
    always_run = timestamp()  
  }  
}  
  
resource "aws_instance" "app" {  
  ami = "ami-0abc123"  
  instance_type = "t3.micro"  
  depends_on = [null_resource.wait]  
}
```



3 DEPENDS_ON

- ✓ Forces a resource (or module) to wait for another to complete.
- ✓ Use sparingly and only when required.

Resource Level

```
resource "aws_db_instance" "db" {  
  ...  
  depends_on = [aws_security_group.db_sg]  
}
```

Module Level

```
module "app" {  
  source = "../modules/app"  
  depends_on = [module.network]  
}
```

⚠ Overusing depends_on can slow down and complicate your deployments.

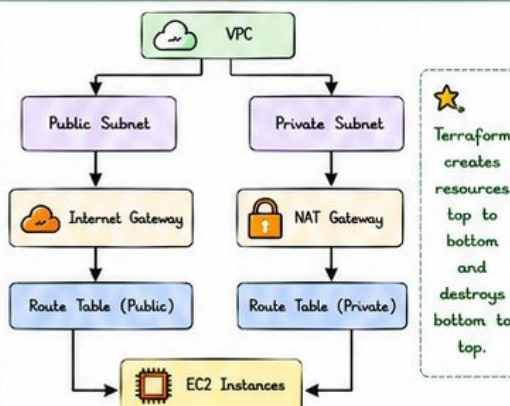
4 RESOURCE GRAPH

- ✓ Terraform builds a graph of all resources and their relationships.
- ✓ It determines the correct order of operations.
- ✓ View it with:

```
terraform graph |  
dot -Tpng graph.png
```



EXAMPLE GRAPH



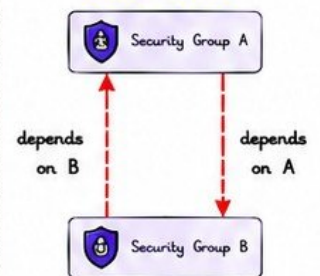
★ Terraform creates resources top to bottom and destroys bottom to top.

5 CIRCULAR DEPENDENCIES

- ✗ Occurs when Resource A depends on B and B depends on A.
- ✗ Terraform cannot determine the order.
- ✗ Leads to plan/apply failure.

⚠ Error example:
"Cycle: aws_security_group.a, aws_security_group.b"

CIRCULAR EXAMPLE

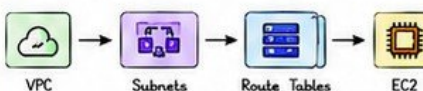


💡 Break the cycle by removing one dependency or using separate rules.

6 PRODUCTION EXAMPLES

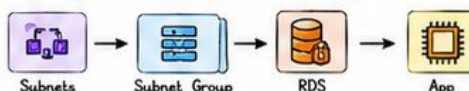
EXAMPLE 1: VPC SETUP

- ✓ Subnets depend on VPC.
- ✓ Route tables depend on subnets & IGW.
- ✓ Instances depend on subnets & SG.



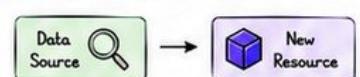
EXAMPLE 2: RDS WITH SG & SUBNET GROUP

- ✓ DB Subnet Group depends on subnets.
- ✓ RDS depends on subnet group & SG.
- ✓ App instance depends on RDS endpoint.



EXAMPLE 3: DATA & RESOURCES

- ✓ Data source reads existing VPC.
- ✓ New resources use data source output.
- ✓ No explicit depends_on needed.



KEY TAKEAWAY

- ✓ Let Terraform handle dependencies automatically.
- ✓ Use depends_on only when absolutely necessary.
- ✓ Understand the graph to design reliable infrastructure.
- ✓ Avoid circular dependencies at all costs.

GOOD PRACTICES

- ✓ Design resources with clear references.
- ✓ Keep modules small and focused.
- ✓ Review the plan and graph before apply.
- ✓ Document custom dependencies.

PRO TIP

A clean dependency graph means fewer surprises in production!

VERIQTA

18. TERRAFORM TROUBLESHOOTING

Problems are normal.
Fixing them like a pro
is what matters!



1 INIT ERRORS

Issues during terraform init.

Common Causes

- No internet / network issues
- Wrong provider version
- Private registry unreachable
- Corrupted .terraform directory



Fix It

- ✓ Check internet and proxy settings.
- ✓ Verify provider source and version.
- ✓ Run: `rm -rf .terraform .terraform.lock.hcl`
- ✓ Re-run: `terraform init -upgrade`.

2 PROVIDER ERRORS

Errors related to providers.

Common Causes

- Wrong credentials / expired keys
- Insufficient permissions
- Unsupported region / API mismatch
- Provider bugs



Fix It

- ✓ Validate credentials:
`aws sts get-caller-identity`
- ✓ Check IAM permissions and policies.
- ✓ Verify region and provider configuration.
- ✓ Try updating provider version.

3 STATE LOCK ISSUES

Another process is using the state file.

Common Causes

- Another terraform process running
- State lock after crash
- CI/CD pipeline overlap
- Manual lock not released



Fix It

- ✓ Check lock info:
`terraform force-unlock <LOCK_ID>`
- ✓ Ensure no other process is running.
- ✓ Avoid manual edits to state files.
- ✓ Use remote state with locking (S3 + DynamoDB).

4 DRIFT (CONFIGURATION DRIFT)

Real infrastructure differs from state.

Common Causes

- Manual changes in console
- External automation tools
- Deleted resources outside Terraform
- Out-of-band updates



Fix It

- ✓ Detect drift: `terraform plan`
- ✓ Review differences carefully.
- ✓ Decide: revert changes or update code.
- ✓ Never ignore drift in production!

5 APPLY FAILURES

terraform apply fails.

Common Causes

- Invalid configuration
- Quota limits
- Insufficient permissions
- Resource conflicts



Fix It

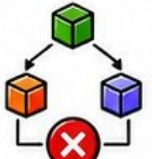
- ✓ Read the full error message.
- ✓ Run `terraform plan` to preview.
- ✓ Check quotas and limits.
- ✓ Fix code, re-run `terraform apply`.

6 DEPENDENCY ERRORS

Resources not created in the right order.

Common Causes

- Missing implicit dependency
- Using data source too early
- Incorrect `depends_on` usage
- Circular dependencies



Fix It

- ✓ Rely on implicit dependencies first.
- ✓ Use `depends_on` only when necessary.
- ✓ Check for circular dependencies.
- ✓ Visualize the graph: `terraform graph`

7 DEBUGGING WORKFLOW (STEP-BY-STEP)

1 Reproduce

Reproduce the issue consistently.



2 Read Error

Read the full error message.
Don't guess!



3 Plan

Run terraform plan to understand differences.



4 Inspect State

Check state:
`terraform state list`
`terraform state show <res>`
`terraform show`



5 Check Logs

Check provider logs and cloud service logs.



6 Fix & Apply

Fix the issue, re-run apply and verify.



USEFUL DEBUG COMMANDS

- `terraform fmt` - Check formatting
- `terraform validate` - Validate code
- `terraform plan -refresh-only` - Check drift

- `TF_LOG=DEBUG terraform plan` - Debug logs
- `terraform providers` - List providers
- `terraform graph | dot -Tpng graph.png` - Graph

- `terraform state list` - List resources
- `terraform state show <addr>` - Show resource
- `terraform taint <addr>` - Mark for recreation



BEST PRACTICES

- ✓ Keep state remote and locked.
- ✓ Use version control for all code.
- ✓ Test changes in non-prod first.
- ✓ Document and review changes.



AVOID THESE

- ✗ Never edit state files manually.
- ✗ Don't ignore errors or warnings.
- ✗ Don't use `depends_on` everywhere.
- ✗ Don't apply without reviewing plan.



PRO TIP

Slow down to speed up!
Good debugging saves
hours of pain later.



VERIQTA

19. COMMON TERRAFORM PRODUCTION MISTAKES

Small mistakes today
become expensive
incidents tomorrow.

Build it right. Run it safe.

1 HARDCODED VALUES

Hardcoding makes code inflexible, insecure, and environment-dependent.

BAD EXAMPLE

```
resource "aws_db_instance" "db" {
  engine      = "postgres"
  username    = "admin"
  password    = "P@ssw0rd123"
  instance_type = "db.t3.large"
}
```



BETTER APPROACH

Use variables, tfvars, or a secrets manager.

- ✓ variable "db_password" {}
- ✓ password = var.db_password



2 LOCAL STATE

Using local state makes collaboration impossible and risks state loss.

BAD EXAMPLE

```
backend "local" {
  path = "terraform.tfstate"
}
```



BETTER APPROACH

Store state remotely (S3, GCS, Azure Blob) with encryption and versioning.

```
backend "s3" {
  bucket      = "tf-state-prod"
  key         = "prod/terraform.tfstate"
  region      = "us-east-1"
  dynamodb_table = "tf-locks"
  encrypt     = true
}
```



3 NO LOCKING

Without state locking, parallel runs can corrupt your state file.

BAD EXAMPLE

S3 backend without DynamoDB state locking.



BETTER APPROACH

Enable state locking with DynamoDB (or your cloud's locking mechanism).

```
backend "s3" {
  ...
  dynamodb_table = "tf-locks"
}
```



4 NO VERSION PINNING

Unpinned versions can break deployments when providers or Terraform change.

BAD EXAMPLE

```
terraform {
  required_providers {
    aws = {}
  }
}
```



BETTER APPROACH

```
terraform {
  required_version = ">= 1.5.0, < 1.7.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.46"
    }
  }
}
```



5 SECRETS IN STATE

Secrets in Terraform state are visible to anyone who can access the state.

BAD EXAMPLE

```
resource "aws_db_instance" "db" {
  password = "SuperSecret123!"
}
```



BETTER APPROACH

Use Secrets Manager, SSM Parameter Store or external secret management.

```
resource "aws_db_instance" "db" {
  password = data.aws_secretsmanager_secret_version.db.password
}
```



6 POOR MODULE DESIGN

Bad modules are hard to reuse, maintain, and scale.

BAD EXAMPLE

- ✗ Too many inputs
- ✗ No outputs
- ✗ Hardcoded values
- ✗ Environment-specific logic



BETTER APPROACH

- ✓ Keep modules small & focused
- ✓ Clear inputs with validation
- ✓ Meaningful outputs
- ✓ No hardcoded values
- ✓ Document usage (README)



7 UNSAFE DESTROY OPERATIONS

Destroying infrastructure without safety checks can cause major outages and data loss.

BAD EXAMPLE

```
terraform destroy
(No review, no backup, runs in prod!)
```



BETTER APPROACH

- ✓ Use workspaces or separate state per environment
- ✓ Run terraform plan and review carefully
- ✓ Use -target only when absolutely necessary
- ✓ Enable deletion protection on critical resources
- ✓ Have backups and runbooks before destroy



QUICK CHECKLIST

- ✓ No hardcoded values
- ✓ Remote state + locking enabled
- ✓ Versions pinned
- ✓ Secrets managed externally
- ✓ Modules are reusable and documented
- ✓ Plan before apply/destroy
- ✓ Backups and runbooks exist

★ BEST PRACTICES

- ✓ Infrastructure as code
- ✓ Review everything
- ✓ Automate safely
- ✓ Test changes in non-prod first
- ✓ Keep it simple and consistent



PRO TIP

Good Terraform is not about writing code. It's about thinking ahead, preventing mistakes, and protecting production.



GOLDEN RULE:

Plan carefully. Code cleanly.
Apply safely. Monitor always.

VERIQTA

20. VOLUME 1 REVIEW & MINI PROJECT

FINAL PAGE!

You made it to the end of Volume 1.

Now it's time to practice like a PRO!

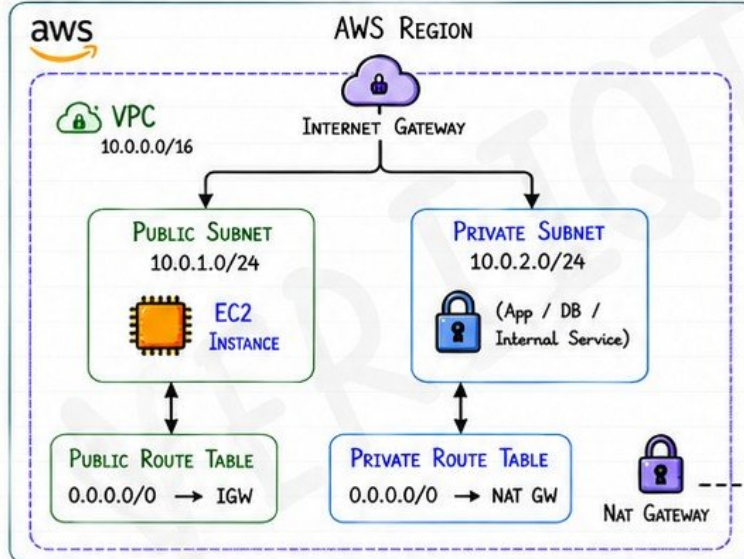


MINI PROJECT: BUILD A PRODUCTION-READY AWS ENVIRONMENT

Use Terraform to build the infrastructure below with best practices.

USE THE FOLLOWING:

- ✓ VPC
- ✓ Public and private subnets
- ✓ Internet Gateway
- ✓ Route tables
- ✓ Security groups
- ✓ EC2 instance
- ✓ Remote state (S3 + DynamoDB)
- ✓ Variables (terraform.tfvars)
- ✓ Outputs



S3 BUCKET
(Remote State)
tf-state-prod

DYNAMODB TABLE
(State Lock)

EXAMPLE VARIABLES (terraform.tfvars)

```
aws_region      = "us-east-1"
vpc_cidr        = "10.0.0.0/16"
public_subnet_cidr = "10.0.1.0/24"
private_subnet_cidr = "10.0.2.0/24"
instance_type   = "t3.micro"
key_name        = "my-key"
environment     = "production"
```



EXAMPLE OUTPUTS (outputs.tf)

```
output "instance_public_ip" {
  value = aws_instance.app.public_ip
}

output "vpc_id" {
  value = aws_vpc.main.id
}

output "public_subnet_id" {
  value = aws_subnet.public.id
}
```



REMOTE STATE SETUP

- 1 Create S3 bucket for state file (e.g., tf-state-prod)
- 2 Enable versioning
- 3 Create DynamoDB table for state locking (e.g., tf-locks)
- 4 Configure backend in terraform { }

```
backend "s3" {
  bucket      = "tf-state-prod"
  key         = "envs/prod/terraform.tfstate"
  region      = "us-east-1"
  dynamodb_table = "tf-locks"
  encrypt     = true
}
```



VALIDATION CHECKLIST

- ✓ terraform init successful
- ✓ terraform fmt clean
- ✓ terraform validate passed
- ✓ terraform plan shows no errors
- ✓ All resources created successfully
- ✓ EC2 instance is reachable (SSH)
- ✓ Route to Internet works (public subnet)
- ✓ Private subnet has no direct Internet access
- ✓ Remote state is stored in S3
- ✓ State locking works (DynamoDB)
- ✓ Outputs return expected values



INTERVIEW QUESTIONS (TEST YOURSELF)

- 1 What is the purpose of `terraform init`?
→ Downloads providers, modules, sets up backend.
- 2 How does Terraform know the order to create resources?
→ It builds a dependency graph.
- 3 What is the difference between `terraform plan` and `terraform apply`?
→ Plan = preview changes, Apply = make changes.
- 4 Why should we store Terraform state in a remote backend?
→ Collaboration, locking, durability, security.
- 5 What happens if two people run Terraform at the same time?
→ State lock prevents concurrent writes.
- 6 How do you prevent accidental resource deletion?
→ Use `prevent_destroy`, reviews, policies, approvals.
- 7 What is the difference between `count` and `for_each`?
→ `count` = by index, `for_each` = by key.
- 8 How do modules improve Terraform code quality?
→ Reusability, maintainability, consistency.



WHAT WE COVERED IN VOLUME 1

- ✓ Terraform Basics
- ✓ Modules
- ✓ Providers & Versions
- ✓ Provisioners
- ✓ Variables & Outputs
- ✓ Dependencies
- ✓ Resources & Arguments
- ✓ Troubleshooting
- ✓ Count & for_each
- ✓ Common Mistakes



PRACTICE.
BUILD.
BREAK. FIX.
REPEAT.

That's how you become a Terraform Expert!



NEXT STEPS

- ★ Finish the mini project
- ★ Experiment with modules
- ★ Explore Terragrunt
- ★ Keep building real projects

KEEP LEVELING UP!

